WIN11 OPTIMIZER

MASTER IMPLEMENTATION DOCUMENT

Terminal UX Design | Tech Stack | Module Architecture | Disclaimer System | Full Implementation

DOCUMENT PROPERTIES	
Classification	INTERNAL — Engineering Use Only
Version	2.0 (Production Release Candidate)
Author	Senior Implementation Engineer | 20 YOE
Standard	Google / Amazon / Microsoft Internal Tooling Practices
Project	Windows 11 Optimizer — Layered Debloat System
Layers Covered	Minimal | Moderate | Ultimate | God Mode
Key Feature	Consent-First Terminal UX with per-tweak disclaimers
Total Tweaks	46 tweaks across 4 layers — all documented
Page Count	30 pages

This document is the single authoritative engineering specification for building, deploying, and maintaining the Windows 11 Optimizer tool. It defines the full tech stack, terminal UX patterns including the consent-first disclaimer system, module-level implementation details, and every disclaimer card shown to users before executing any tweak.

TABLE OF CONTENTS

01 | EXECUTIVE SUMMARY & PROJECT CONTEXT

The Windows 11 Optimizer is a production-grade terminal application designed to systematically reduce telemetry, background resource consumption, and UI noise on Windows 11 systems. Built to the same engineering standards used in internal tooling at Google, Amazon, and Microsoft, it emphasizes three core principles: **safety**, **reversibility**, and **informed consent**.

PROJECT MANDATE

DIMENSION	REQUIREMENT
Safety	Zero damage to Windows Update, Defender, Servicing Stack, or core runtimes. Ever.
Reversibility	Every change has a documented, tested, single-command rollback.
Consent	User sees a plain-English ~100 word summary before EVERY tweak executes.
Layered Control	4 escalating layers. User chooses depth. Never forced to go further.
Transparency	Full logging. Diff viewer. Before/after metrics. No black boxes.
Production Grade	Error handling, retry logic, state machine, idempotent operations.
Distribution	Single .ps1 file. No internet required post-download. No dependencies.

WHAT MAKES THIS DIFFERENT FROM RANDOM DEBLOAT SCRIPTS

RANDOM SCRIPTS	WIN11 OPTIMIZER
Run blind — no explanation of what happens	Every tweak has a 100-word disclaimer
No rollback mechanism	Auto-generated rollback script per layer
No state tracking	JSON state machine with applied layer history
No testing framework	Built-in verification suite with pass/fail
Often breaks Windows Update	Update stack is an immutable protected zone
No dry-run mode	Full dry-run: shows commands without executing
Monolithic script — all or nothing	Per-tweak toggle — skip what you don't want
No logging	Append-only audit log with timestamps

02 | TECH STACK — FULL SPECIFICATION

The tool is written entirely in PowerShell 5.1+ with no external dependencies beyond what ships with Windows 11. The TUI layer uses ANSI escape codes and Write-Host for rendering. No .NET frameworks beyond what PowerShell loads natively.

CORE RUNTIME

COMPONENT	TECHNOLOGY	VERSION	REASON
Shell / Runtime	PowerShell	5.1+	Ships with Win11, no install needed
TUI Rendering	ANSI Escape Codes	VT100/220	Works in Windows Terminal + ConHost
Config Storage	JSON (ConvertTo-Json)	Built-in	Human-readable, no DB needed
Registry Access	Reg.exe + PS Cmdlets	Built-in	Native OS-level config access
Service Control	sc.exe + Set-Service	Built-in	Reliable service management
Firewall	NetFirewallRule CmdLt	Built-in	WFP-level blocking
AppX Management	Remove-AppxPackage	Built-in	Official Microsoft UWP removal API
Task Scheduler	schtasks.exe	Built-in	Stable CLI for scheduled tasks
Logging	Add-Content (PS)	Built-in	Append-only, thread-safe-ish
Backup/Snapshot	Export-Csv + reg export	Built-in	Zero-dep state export
Terminal	Windows Terminal	1.0+	Recommended for best ASCII render

OPTIONAL COMPANION TOOLS (User Installs)

TOOL	PURPOSE	INSTALL
Everything (voidtools)	Replace WSearch for fast file search	winget install voidtools.Everything
Macrium Reflect Free	Full disk image before God Mode	macrium.com/reflectfree
WinUtil (CTT)	Inspiration / reference for patterns	christitustech.dev/winutil
simplewall	WFP GUI for advanced firewall control	henrypp.org/product/simplewall
Process Hacker 2	Deep process/service visibility	processhacker.sourceforge.io
autoruns (SysInternals)	Full autostart audit	winget install sysinternals.autoruns

POWERSHELL VERSION COMPATIBILITY

PS VERSION	COMPATIBILITY	NOTES
PS 5.1	FULL	Ships with Windows 11 — primary target
PS 7.x	FULL	Faster, cross-platform — recommended for devs
PS 4.0	PARTIAL	Missing some cmdlets — upgrade recommended
PS 3.0	NOT SUPPORTED	Too old — missing critical cmdlets

03 | ARCHITECTURE & DIRECTORY STRUCTURE

MODULE DEPENDENCY GRAPH

All modules are loaded by the main orchestrator (WinOptimizer.ps1). Modules are independent — each can be sourced and tested in isolation. This follows the Single Responsibility Principle used in Google's internal tooling.

```
WinOptimizer.ps1  (Orchestrator + TUI Engine)
|
+-- modules/Snapshot.psm1          # State export before any change
|      Export-ServiceBaseline()
|      Export-AppxBaseline()
|      Export-FirewallBaseline()
|      Create-RestorePoint()
|
+-- modules/Disclaimer.psm1        # THE CONSENT SYSTEM (core UX feature)
|      Show-DisclaimerCard()        # Renders 100-word summary card
|      Get-UserConsent()            # Returns: YES / NO / RUNALL / EXIT
|      Set-GlobalConsent()          # Stores 'run all' state for session
|
+-- modules/Layer_Minimal.psm1     # 16 tweaks, zero risk
+-- modules/Layer_Moderate.psm1    # 14 tweaks, low risk
+-- modules/Layer_Ultimate.psm1    #  9 tweaks, medium risk
+-- modules/Layer_GodMode.psm1     # 17 tweaks, high risk
|      Each exports:
|        Invoke-[Layer]Tweaks($DryRun, $SkipList)
|        Undo-[Layer]Tweaks()
|        Get-[Layer]TweakList()
|
+-- modules/Rollback.psm1          # Rollback engine
|      Invoke-Rollback($Layer)
|      Invoke-RollbackSingle($TweakId)
|      Register-RollbackCmd($Layer, $Id, $Cmd)
|
+-- modules/Verify.psm1            # Post-tweak verification
|      Test-SystemIntegrity()
|      Test-LayerApplied($Layer)
|      Get-VerifyReport()
|
+-- modules/Status.psm1            # Live metrics panel
|      Get-SystemMetrics()
|      Show-StatusPanel()
|
+-- modules/Logger.psm1            # Audit trail
       Write-Log($Level, $Msg)
       Export-SessionLog()
```

FULL DIRECTORY STRUCTURE

```
C:\WinOptimizer\
|
|-- WinOptimizer.ps1                  [MAIN ENTRY — run this as Admin]
|-- INSTALL.md                        [Quick start instructions]
|-- VERSION                           [Semver string: 2.0.0]
|
|-- modules\
|   |-- Disclaimer.psm1               [CONSENT SYSTEM]
|   |-- Snapshot.psm1
|   |-- Layer_Minimal.psm1
|   |-- Layer_Moderate.psm1
|   |-- Layer_Ultimate.psm1
```

```
|   |-- Layer_GodMode.psm1
|   |-- Rollback.psm1
|   |-- Verify.psm1
|   |-- Status.psm1
|   +-- Logger.psm1
|
|-- config\
|   |-- applied_layers.json            [State machine — which layers applied]
|   |-- user_prefs.json                [Skip list, color theme, verbosity]
|   +-- protected_services.json        [Immutable: services never touched]
|
|-- snapshots\                         [Auto-created before each layer]
|   |-- baseline_services.csv
|   |-- baseline_appx.csv
|   |-- baseline_tasks.csv
|   +-- baseline_firewall.csv
|
|-- rollback\                          [Auto-generated inverse scripts]
|   |-- rollback_minimal.ps1
|   |-- rollback_moderate.ps1
|   |-- rollback_ultimate.ps1
|   +-- rollback_godmode.ps1
|
|-- logs\
|   |-- session_YYYYMMDD_HHmm.log      [Per-session append-only audit log]
|   +-- verify_YYYYMMDD_HHmm.csv       [Verification scan results]
|
+-- assets\
    +-- ascii_logo.txt                 [ASCII art header loaded at startup]
```

04 | CONSENT-FIRST TERMINAL UX PHILOSOPHY

The single most important design decision in this tool is the **Consent-First UX**. Every tweak is gated behind a human-readable summary. No command fires without the user explicitly saying YES. This is non-negotiable and is modeled after patterns used in production deployment tools at Google (Borg), Amazon (Apollo), and HashiCorp Terraform.

THE FOUR CONSENT MODES

KEY	ACTION	BEHAVIOR	WHEN TO USE
Y	YES	Apply this single tweak, then show next	Reviewing one-by-one
N	NO / SKIP	Skip this tweak, move to next	Don't need this tweak
A	RUN ALL	Apply all remaining tweaks in layer without prompting	Trusted mode after review
D	DRY RUN	Show what WOULD happen — no changes made	First-time preview
Q / EXIT	QUIT	Stop everything, save state, exit cleanly	Changed mind

DISCLAIMER CARD ANATOMY

Each disclaimer card shown in the terminal has exactly 5 zones. Together they give the user everything they need to make an informed decision in under 30 seconds.

ZONE	CONTENT	MAX LENGTH
Z1	HEADER: Layer name + tweak number + tweak title	1 line
Z2	PLAIN ENGLISH SUMMARY: What this does in 80-100 words, no jargon	~100 words
Z3	WHAT CHANGES: The specific files/services/keys affected	1 line
Z4	REVERSIBLE: Yes/Partial/No + how to undo if needed	1 line
Z5	PROMPT: > Proceed? [Y]es [N]o [A]ll [D]ry-run [Q]uit	1 line, blinking cursor

UX RULES — NEVER VIOLATE THESE

RULE 1: Never execute a command without first showing its disclaimer card.

RULE 2: Summary text must use plain English. No technical jargon in Z2.

RULE 3: If user chooses RUN ALL, still log each tweak as if individually confirmed.

RULE 4: In God Mode layer, RUN ALL requires a second confirmation: type 'GODMODE' to confirm.

RULE 5: Dry-run mode must print every command that WOULD run, in green italic.

RULE 6: After any failure, the loop STOPS — user must explicitly continue or rollback.

RULE 7: Session state (what was applied) must be saved even on CTRL+C exit.

RULE 8: Summary must mention the rollback procedure in 1 sentence.

05 | DISCLAIMER SYSTEM — DESIGN & IMPLEMENTATION

DISCLAIMER MODULE CODE (Disclaimer.psm1)

```
function Show-DisclaimerCard {
    param(
        [int]   $Number,
        [string]$Layer,          # MINIMAL | MODERATE | ULTIMATE | GOD MODE
        [string]$TweakName,
        [string]$Summary,        # 80-100 word plain English description
        [string]$WhatChanges,    # "Disables 2 services: DiagTrack, dmwappush"
        [string]$Reversible,     # "YES — run rollback_minimal.ps1"
        [string]$RiskLevel       # ZERO | LOW | MEDIUM | HIGH
    )

    $layerColors = @{
        "MINIMAL"  = "Green";  "MODERATE" = "Yellow";
        "ULTIMATE" = "DarkYellow"; "GOD MODE" = "Red"
    }
    $lc = $layerColors[$Layer]

    Clear-Host   # Fresh screen for each disclaimer
    # ■■ ASCII Header ■■
    Write-Host ""
    Write-Host "  +$('■' * 66)+" -ForegroundColor DarkGray
    Write-Host "  |  DISCLAIMER #$($Number.ToString('00'))  [$Layer]  —  $TweakName" -NoNewline
    Write-Host (" " * (65 - 18 - $Layer.Length - $TweakName.Length)) -NoNewline
    Write-Host "|" -ForegroundColor DarkGray
    Write-Host "  +$('■' * 66)+" -ForegroundColor DarkGray

    # ■■ Summary block ■■
    Write-Host ""
    Write-Host "  WHAT THIS DOES:" -ForegroundColor $lc
    Write-Host ""
    $words = $Summary -split ' '
    $line = "    "; $lw = 0
    foreach ($w in $words) {
        if ($lw + $w.Length + 1 -gt 66) {
            Write-Host $line -ForegroundColor White
            $line = "    $w "; $lw = $w.Length + 5
        } else { $line += "$w "; $lw += $w.Length + 1 }
    }
    if ($line.Trim()) { Write-Host $line -ForegroundColor White }

    # ■■ Meta row ■■
    Write-Host ""
    Write-Host "  AFFECTS   : " -NoNewline -ForegroundColor DarkGray
    Write-Host $WhatChanges -ForegroundColor Cyan
    Write-Host "  REVERSIBLE: " -NoNewline -ForegroundColor DarkGray
    $rcolor = if ($Reversible -match "^YES") {"Green"} else {"Yellow"}
    Write-Host $Reversible -ForegroundColor $rcolor
    Write-Host "  RISK LEVEL: " -NoNewline -ForegroundColor DarkGray
    $rkc = @{"ZERO"="Green";"LOW"="Yellow";"MEDIUM"="DarkYellow";"HIGH"="Red"}[$RiskLevel]
    Write-Host $RiskLevel -ForegroundColor $rkc
    Write-Host ""
    Write-Host "  +$('■' * 66)+" -ForegroundColor DarkGray

    # ■■ Prompt ■■
    Write-Host "  > Proceed? [Y] Yes  [N] Skip  [A] Run All  [D] Dry-run  [Q] Quit" -ForegroundColor Ye
llow
    Write-Host "  > " -NoNewline -ForegroundColor Green
}

function Get-UserConsent {
```

```
    param([string]$Layer, [bool]$GlobalRunAll = $false)

    if ($GlobalRunAll) {
        # Special: God Mode requires extra confirmation even in RunAll
        if ($Layer -eq "GOD MODE") { return "YES" }
        return "YES"
    }

    $key = $Host.UI.RawUI.ReadKey("NoEcho,IncludeKeyDown").Character.ToString().ToUpper()

    return switch ($key) {
        "Y" { "YES" }
        "N" { "SKIP" }
        "A" {
            if ($Layer -eq "GOD MODE") {
                Write-Host "`n  [GOD MODE] Type 'GODMODE' to confirm run-all: " -NoNewline -ForegroundC
olor Red
                $confirm = Read-Host
                if ($confirm -eq "GODMODE") { "RUNALL" } else { "SKIP" }
            } else { "RUNALL" }
        }
        "D" { "DRYRUN" }
        "Q" { "EXIT" }
        default { "SKIP" }
    }
}
```

06 | MASTER PROMPT PATTERN — CODE TEMPLATE

Every tweak function follows this exact pattern. This is the implementation contract — no tweak is allowed to deviate from it.

```
function Invoke-Tweak {
    param(
        [int]   $TweakNumber,
        [string]$Layer,
        [string]$TweakName,
        [string]$Summary,      # ~100 words plain English
        [string]$WhatChanges,
        [string]$Reversible,
        [string]$RiskLevel,
        [switch]$DryRun,
        [ref]   $GlobalRunAll
    )

    # ■■ 1. Show disclaimer ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    Show-DisclaimerCard -Number $TweakNumber -Layer $Layer `
        -TweakName $TweakName -Summary $Summary `
        -WhatChanges $WhatChanges -Reversible $Reversible -RiskLevel $RiskLevel

    # ■■ 2. Get consent ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    $consent = Get-UserConsent -Layer $Layer -GlobalRunAll $GlobalRunAll.Value
    Write-Host ""

    switch ($consent) {
        "EXIT"   { Save-SessionState; exit 0 }
        "SKIP"   { Write-Log -Level SKIP -Msg "SKIPPED: $TweakName"; return }
        "RUNALL" { $GlobalRunAll.Value = $true }
        "DRYRUN" { $DryRun = $true }
    }

    # ■■ 3. Create restore point (first tweak only) ■■■■■■■■■■■
    if ($TweakNumber -eq 1) {
        Write-Log -Level INFO -Msg "Creating restore point for $Layer layer..."
        Checkpoint-Computer -Description "pre-$($Layer.ToLower().Replace(' ','-'))" `
            -RestorePointType MODIFY_SETTINGS -ErrorAction SilentlyContinue
    }

    # ■■ 4. Execute ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    if ($DryRun) {
        Write-Host "  [DRY-RUN] Commands that WOULD execute:" -ForegroundColor Cyan
        # ... print commands here
        Write-Log -Level DRYRUN -Msg "DRY-RUN: $TweakName"
        return
    }

    try {
        Write-Host "  Applying: $TweakName ..." -NoNewline -ForegroundColor Cyan
        # ■■ ACTUAL COMMANDS GO HERE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        # ... tweak-specific implementation ...

        Write-Host " [OK]" -ForegroundColor Green
        Write-Log -Level SUCCESS -Msg "APPLIED: $TweakName"
        Register-RollbackCmd -Layer $Layer -TweakId $TweakNumber `
            -Command ""
    }
    catch {
        Write-Host " [FAIL]" -ForegroundColor Red
        Write-Log -Level ERROR -Msg "FAILED: $TweakName | Error: $_"
        Write-Host "  Error: $_" -ForegroundColor Red
        Write-Host "  Press any key to attempt rollback, or Q to skip rollback..." -ForegroundColor Yel
```

```
low
        $k = $Host.UI.RawUI.ReadKey("NoEcho,IncludeKeyDown").Character.ToString().ToUpper()
        if ($k -ne "Q") { Invoke-RollbackSingle -TweakId $TweakNumber -Layer $Layer }
    }
}
```

07 | LAYER 1: MINIMAL — ALL DISCLAIMER CARDS

These are the exact disclaimer cards shown to the user before each Minimal layer tweak. Each summary is ~100 words in plain English. No jargon. No fear-mongering. Honest.

DISCLAIMER #1 [MINIMAL] — Disable DiagTrack Telemetry Service

Windows runs a background service called DiagTrack that constantly collects data about how you use your computer — app crashes, featu

CHANGES: Stops + disables 1 service: DiagTrack (Connected User Experiences) **REVERSIBLE: YES — sc config DiagTrack s**

ZERO

DISCLAIMER #2 [MINIMAL] — Disable Background App Access (UWP Global)

Windows 11 allows apps like Mail, Calendar, Weather, and News to run invisible background tasks — syncing, refreshing content, downloa

CHANGES: Sets registry: BackgroundAccessApplications > GlobalUserDisabled = 1 **REVERSIBLE: YES — delete GlobalUserDis**

ZERO

DISCLAIMER #3 [MINIMAL] — Disable Windows Widgets (WebView2 Panel)

The Widgets panel on the left of your taskbar is not just a simple news feed. It runs a full mini-browser (WebView2) in the background at all

CHANGES: Disables Widgets via GPO registry: AllowNewsAndInterests = 0 **REVERSIBLE: YES — delete AllowNewsAnd**

ZERO

DISCLAIMER #4 [MINIMAL] — Remove Start Menu Ads and Suggestions

Microsoft embeds advertisements and sponsored app suggestions directly inside your Start menu. These are called 'Recommended' apps a

CHANGES: Sets 4 ContentDeliveryManager registry values to 0 **REVERSIBLE: YES — set values back to 1 i**

ZERO

DISCLAIMER #5 [MINIMAL] — Disable Windows Spotlight Lock Screen

Windows Spotlight is the feature that downloads a new Bing photo every day and displays it on your lock screen, along with trivia and tips. V

CHANGES: Sets lock screen type to Picture via registry CloudContent policy **REVERSIBLE: YES — Settings > Personaliz**

ZERO

DISCLAIMER #6 [MINIMAL] — Disable Edge Startup Boost and Background Processes

Microsoft Edge continues running in the background even after you close every browser window. It does this to make the next time you ope

CHANGES: Disables Edge Startup Boost + Continue Running Background Extensions **REVERSIBLE: YES — Edge Settings > Syst**

ZERO

DISCLAIMER #7 [MINIMAL] — Disable Xbox Game Bar

Xbox Game Bar is Microsoft's built-in gaming overlay — press Win+G to open it. It provides game recording, performance HUDs, and Xbox

CHANGES: Sets AppCaptureEnabled = 0 in registry, disables Game Bar services **REVERSIBLE: YES — Settings > Gaming >**

ZERO

DISCLAIMER #8 [MINIMAL] — Disable Delivery Optimization (P2P Updates)

Windows uses your internet connection to upload Windows Update files to other Windows PCs — both on your local network and across the

CHANGES: Sets DeliveryOptimization DODownloadMode = 0 in registry **REVERSIBLE: YES — Settings > Windows U**

ZERO

DISCLAIMER #9 [MINIMAL] — Disable Windows Tips and Feature Suggestions

Windows frequently interrupts you with pop-up notifications suggesting you 'finish setting up your PC', try new features, use OneDrive, or sw

CHANGES: Sets 4 SubscribedContent registry values to 0 (notification ads) **REVERSIBLE: YES — set SubscribedConte**

ZERO

DISCLAIMER #10 [MINIMAL] — Reduce Search Index Scope (Exclude Dev Folders)

Windows Search Indexer runs continuously to index every file on your PC so search results appear instantly. This is great for Documents ar

CHANGES: Adds excluded folders to Windows Search index via Settings **REVERSIBLE: YES — Settings > Privacy > S**

ZERO

08 | LAYER 2: MODERATE — ALL DISCLAIMER CARDS

Moderate layer tweaks involve registry policies, service disabling, and UWP app removal. Risk increases slightly — some changes require Store reinstalls to reverse. All summaries below are the exact text displayed to users.

DISCLAIMER #1 [MODERATE] — Set Telemetry Level to Minimum via Policy Registry

We're setting a Group Policy registry key that tells Windows to collect the absolute minimum amount of diagnostic data allowed by Microsoft

CHANGES: Sets HKLM: AllowTelemetry = 0 + HKCU: AdvertisingInfo Enabled = 0 **REVERSIBLE: YES — delete HKLM:\SOFTW**

LOW

DISCLAIMER #2 [MODERATE] — Disable Xbox Gaming Services (4 Services)

Windows installs four Xbox-related background services even if you never use Xbox: XblAuthManager handles Xbox account authentication

CHANGES: Stops + disables: XblAuthManager, XblGameSave, XboxGipSvc, XboxNetApiSvc **REVERSIBLE: YES — Set-Service each to M**

LOW

DISCLAIMER #3 [MODERATE] — Disable Fax, MapsBroker, RetailDemo, WMPNetworkSvc

Four services that most people never use but Windows runs anyway: Fax enables old-school fax machine support, MapsBroker handles off

CHANGES: Stops + disables: Fax, MapsBroker, RetailDemo, WMPNetworkSvc **REVERSIBLE: YES — Set-Service each bac**

ZERO

DISCLAIMER #4 [MODERATE] — Disable CEIP Scheduled Tasks

Windows runs several hidden scheduled tasks as part of the Customer Experience Improvement Program. These tasks wake up periodicall

CHANGES: Disables 2 scheduled tasks: CEIP Consolidator + AppExperience ProgramDataUpdater **REVERSIBLE: YES — Enable-ScheduledTas**

LOW

DISCLAIMER #5 [MODERATE] — Remove Xbox, 3D Viewer, Solitaire, GetHelp UWP Apps

Windows 11 ships with several pre-installed apps you likely never use: Microsoft Solitaire Collection with its ad-heavy gaming platform, Micr

CHANGES: Removes AppX packages: Xbox apps, 3D Viewer, Solitaire, GetHelp, StickyNotes **REVERSIBLE: PARTIAL — reinstall from Mi**

LOW

DISCLAIMER #6 [MODERATE] — Disable SysMain (Superfetch) on SSD Systems

SysMain, formerly known as Superfetch, is a Windows service that tries to predict which applications you'll open next and pre-loads them in

CHANGES: Stops + disables service: SysMain (Superfetch/Prefetch background caching) **REVERSIBLE: YES — Set-Service SysMain**

LOW

DISCLAIMER #7 [MODERATE] — Add Defender Exclusions for Dev and Data Folders

Windows Defender scans every file your programs touch in real time. This is great for security but devastating for development workflows. W

CHANGES: Adds ExclusionPath entries to Windows Defender via Add-MpPreference

REVERSIBLE: YES — Remove-MpPreferenc

LOW

DISCLAIMER #8 [MODERATE] — Uninstall OneDrive (Optional — Read Before Confirming)

OneDrive is Microsoft's cloud sync service that automatically backs up your Desktop, Documents, and Pictures to the cloud. If you actively u

CHANGES: Runs OneDriveSetup.exe /uninstall — removes OneDrive client only

REVERSIBLE: YES — winget install Microso

LOW

DISCLAIMER #9 [MODERATE] — Disable Edge Scheduled Background Tasks

Microsoft Edge installs scheduled tasks in Windows Task Scheduler that run Edge-related processes on a timer even when you've never op

CHANGES: Disables Edge-related scheduled tasks found in Task Scheduler

REVERSIBLE: YES — Enable-ScheduledTas

LOW

DISCLAIMER #10 [MODERATE] — Disable dmwappushservice (Device Management Push)

The dmwappushservice (Device Management WAP Push Message Routing Service) handles push messages for Mobile Device Manageme

CHANGES: Stops + disables: dmwappushservice (MDM push routing)

REVERSIBLE: YES — Set-Service dmwappu

LOW

09 | LAYER 3: ULTIMATE — ALL DISCLAIMER CARDS

Ultimate layer involves firewall rules, system-wide app de-provisioning, and hardware-adjacent changes. A disk image backup is strongly recommended before this layer. Risk level escalates — read each card carefully.

WARNING: Before starting this layer the tool will verify: (1) A System Restore point was created, (2) A disk image exists on an external drive. If either is missing, the tool will PAUSE and prompt you to create them before continuing.

DISCLAIMER #1 [ULTIMATE] — Block Telemetry Endpoints via Windows Firewall (WFP)

We're going further than just disabling the telemetry service — we're adding outbound firewall rules that block network traffic to known Micro

CHANGES: Creates outbound block rules in Windows Firewall for telemetry IPs

REVERSIBLE: YES — Remove-NetFirewallR

MEDIUM

DISCLAIMER #2 [ULTIMATE] — Un-provision AppX Packages for ALL Users

The Moderate layer removed these apps for your current user account. This goes further — it removes the provisioned package that would

CHANGES: Removes provisioned AppX packages for all future user accounts

REVERSIBLE: PARTIAL — re-register via A

MEDIUM

DISCLAIMER #3 [ULTIMATE] — Disable Windows Error Reporting Service (WerSvc)

When an application crashes, Windows Error Reporting captures a memory dump and sends a crash report to Microsoft. This data helps Mi

CHANGES: Stops + disables: WerSvc (Windows Error Reporting Service)

REVERSIBLE: YES — Set-Service WerSvc -

LOW

DISCLAIMER #4 [ULTIMATE] — Disable Hibernation and Free Hiberfil.sys Space

Hibernation saves your entire RAM contents to a file called hiberfil.sys so your PC can power off completely and restore exactly where you l

CHANGES: Runs powercfg -h off, deletes hiberfil.sys, disables Fast Startup

REVERSIBLE: YES — powercfg -h on (recre

LOW

DISCLAIMER #5 [ULTIMATE] — Prevent Windows Update from Auto-Installing Drivers

Windows Update automatically downloads and installs hardware drivers — GPU drivers, network adapter drivers, USB drivers — whenever

CHANGES: Sets GPO: ExcludeWUDriversInQualityUpdate = 1 via registry

REVERSIBLE: YES — delete ExcludeWUDri

MEDIUM

DISCLAIMER #6 [ULTIMATE] — Deep Disable of All Remaining CEIP Telemetry Tasks

The Moderate layer disabled the two most active CEIP scheduled tasks. This step goes through the entire Task Scheduler and disables eve

CHANGES: Disables all scheduled tasks matching: Customer Experience, App Experience, Telemetry

REVERSIBLE: YES — Enable-ScheduledTas

LOW

DISCLAIMER #7 [ULTIMATE] — Remove OEM Bloatware Applications (Keep Drivers)

Your laptop manufacturer (Acer, Dell, HP, Lenovo, etc.) pre-installs a collection of their own apps: trial antivirus, photo editors, update utilitie

CHANGES: Uninstalls OEM application packages (not drivers) via Remove-Package

REVERSIBLE: PARTIAL — reinstall from ma

MEDIUM

DISCLAIMER #8 [ULTIMATE] — Configure Pagefile for Fixed Size or Secondary Drive

The pagefile is Windows' overflow memory — when your RAM fills up, Windows spills data into this file on your SSD. By default Windows a

CHANGES: Sets pagefile to fixed size or relocates to secondary drive via System Properties

REVERSIBLE: YES — System Properties >

MEDIUM

DISCLAIMER #9 [ULTIMATE] — Disable WSearch Indexer Completely

We previously just excluded folders from Windows Search. This step disables the entire Windows Search indexing service. Start menu sear

CHANGES: Stops + disables: WSearch service entirely

REVERSIBLE: YES — sc config WSearch st

MEDIUM

10 | LAYER 4: GOD MODE — ALL DISCLAIMER CARDS

God Mode is the deepest layer. A confirmed disk image backup is MANDATORY. The tool will refuse to proceed without verifying it exists. Run All in this layer requires typing 'GODMODE' as a second confirmation.

MANDATORY PRE-CHECKS: (1) Full disk image on external drive — verified. (2) System Restore point created — verified. (3) Separate local admin account exists — confirmed. If any check fails, God Mode will not start.

DISCLAIMER #1 [GOD MODE] — Block All Consumer Experience Features (CDM System)

The ContentDeliveryManager is Microsoft's system for pushing sponsored apps, suggestions, and promotional content onto your PC withou

CHANGES: Sets HKLM CloudContent: DisableConsumerFeatures = 1 **REVERSIBLE: YES — delete DisableConsu**

LOW

DISCLAIMER #2 [GOD MODE] — Disable Windows Copilot and AI Recall Features

Windows 11 23H2 and later includes Windows Copilot — an AI assistant panel — and in some builds, Windows Recall, which takes screens

CHANGES: Sets TurnOffWindowsCopilot = 1, disables Recall indexing service **REVERSIBLE: YES — delete TurnOffWindo**

LOW

DISCLAIMER #3 [GOD MODE] — Disable Windows OneSyncSvc (Account Settings Sync)

OneSyncSvc synchronizes your Windows settings — theme, passwords, language preferences, accessibility settings — across multiple Wi

CHANGES: Stops + disables: OneSyncSvc (Microsoft account cross-device sync) **REVERSIBLE: YES — Set-Service OneSync**

MEDIUM

DISCLAIMER #4 [GOD MODE] — Disable Print Spooler Service

The Print Spooler manages all printing on Windows — it queues print jobs, talks to printer drivers, and handles network print sharing. If you

CHANGES: Stops + disables: Spooler service (printing infrastructure) **REVERSIBLE: YES — sc config Spooler sta**

LOW

DISCLAIMER #5 [GOD MODE] — Disable WMPNetworkSvc (Media Streaming DLNA)

WMPNetworkSvc is the Windows Media Player Network Sharing Service — it broadcasts your media library over your local network using t

CHANGES: Stops + disables: WMPNetworkSvc (DLNA media sharing server) **REVERSIBLE: YES — Set-Service WMPNetw**

LOW

DISCLAIMER #6 [GOD MODE] — Disable RetailDemo Service

RetailDemo is a Windows service designed for store display PCs — the kind you see in Best Buy or a Microsoft Store where Windows plays

CHANGES: Stops + disables: RetailDemo service (store display demonstration mode) **REVERSIBLE: YES — Set-Service RetailDe**

ZERO

DISCLAIMER #7 [GOD MODE] — Remove Full Xbox Ecosystem (Apps + Services)

This is a more complete removal than the Moderate layer's partial Xbox cleanup. We're removing all remaining Xbox AppX packages and se

CHANGES: Removes all Xbox AppX + disables XblAuthManager, XblGameSave, XboxNetApiSvc **REVERSIBLE: PARTIAL — reinstall Xbox ap**

MEDIUM

DISCLAIMER #8 [GOD MODE] — Disable Background App Infrastructure Globally

This deepens the Minimal layer's background app restriction by applying it at a more persistent registry level and ensures it applies to all ap

CHANGES: Sets GlobalUserDisabled = 1 more permanently via policy-level registry path **REVERSIBLE: YES — delete GlobalUserDis**

MEDIUM

DISCLAIMER #9 [GOD MODE] — Disable Windows Error Reporting (WER) Completely

We may have already partially addressed this. This step performs a complete WER shutdown — stopping the service, disabling it, and also

CHANGES: Stops + disables WerSvc + sets WER policy registry to disabled mode **REVERSIBLE: YES — Start-Service WerSvc**

LOW

DISCLAIMER #10 [GOD MODE] — Disable Delivery Optimization via Registry (Full Block)

The Minimal layer turned off P2P delivery optimization via Settings. This goes deeper — it sets the registry value directly so the policy survi

CHANGES: Sets DODownloadMode = 0 via registry key (policy-level persistent) **REVERSIBLE: YES — delete DODownloadM**

ZERO

DISCLAIMER #11 [GOD MODE] — Disable MapsBroker Completely

MapsBroker is the Windows Maps background service that handles automatic offline map downloads. When you open the Maps app and sa

CHANGES: Stops + disables: MapsBroker service (offline Maps sync daemon) **REVERSIBLE: YES — Set-Service MapsBro**

ZERO

DISCLAIMER #12 [GOD MODE] — Disable SysMain Completely

Earlier layers may have addressed SysMain partially. This applies the definitive disable — stopping the service and setting it to disabled via

CHANGES: Stops + disables SysMain via sc.exe (most reliable method) **REVERSIBLE: YES — sc config SysMain sta**

LOW

DISCLAIMER #13 [GOD MODE] — Disable Windows Spotlight Permanently via Policy

We disabled Spotlight in the Minimal layer via user settings. Policy-level blocking via registry is more persistent — it survives user account c

CHANGES: Sets HKCU\Software\Policies\Microsoft\Windows\CloudContent DisableWindowsSpotlight **REVERSIBLE: YES — delete DisableWindow**

ZERO

DISCLAIMER #14 [GOD MODE] — Disable Widgets at Policy Level (AllowNewsAndInterests)

The Minimal layer disabled Widgets via a registry value. This confirms the same setting at the Group Policy equivalent registry path which is

CHANGES: Sets HKLM\SOFTWARE\Policies\Microsoft\Dsh AllowNewsAndInterests = 0 **REVERSIBLE: YES — delete AllowNewsAnd**

LOW

DISCLAIMER #15 [GOD MODE] — Block Additional Telemetry IPs via Firewall

Building on the Ultimate layer's firewall rules, we add additional known telemetry endpoint IP addresses to the outbound block list. These in

CHANGES: Adds 5-8 additional outbound block rules for telemetry IP ranges **REVERSIBLE: YES — Remove-NetFirewallR**

MEDIUM

DISCLAIMER #16 [GOD MODE] — Disable DiagTrack + Full CEIP Task Suite

This is the final and most comprehensive telemetry sweep. It goes through the full scheduled task library and disables every task that conta

CHANGES: Disables all telemetry/CEIP scheduled tasks found across all task paths **REVERSIBLE: YES — Enable-ScheduledTas**

LOW

DISCLAIMER #17 [GOD MODE] — Disable OneSyncSvc (Final Pass — Policy Level)

This is the definitive, policy-level disabling of OneSyncSvc that supersedes any previous attempts. OneSyncSvc is the Microsoft account sy

CHANGES: Stops + disables OneSyncSvc via sc.exe + sets policy registry override **REVERSIBLE: YES — sc config OneSyncSv**

MEDIUM

11 | TUI ENGINE IMPLEMENTATION

MAIN LOOP ARCHITECTURE

```
# WinOptimizer.ps1 — Main TUI Entry Point

#Requires -RunAsAdministrator
Set-StrictMode -Version Latest
$ErrorActionPreference = "Stop"

# Load all modules
$modulePath = "$PSScriptRoot\modules"
Import-Module "$modulePath\Logger.psm1"       -Force
Import-Module "$modulePath\Snapshot.psm1"     -Force
Import-Module "$modulePath\Disclaimer.psm1"   -Force
Import-Module "$modulePath\Rollback.psm1"     -Force
Import-Module "$modulePath\Verify.psm1"       -Force
Import-Module "$modulePath\Status.psm1"       -Force
Import-Module "$modulePath\Layer_Minimal.psm1"  -Force
Import-Module "$modulePath\Layer_Moderate.psm1" -Force
Import-Module "$modulePath\Layer_Ultimate.psm1" -Force
Import-Module "$modulePath\Layer_GodMode.psm1"  -Force

# Init
Write-Log -Level INFO -Msg "Session started"
$globalState = Get-Content "$PSScriptRoot\config\applied_layers.json" | ConvertFrom-Json

function Show-MainMenu {
    Clear-Host
    $logo = Get-Content "$PSScriptRoot\assets\ascii_logo.txt" -Raw
    Write-Host $logo -ForegroundColor Green
    Write-Host ""
    Write-Host "  ╔══════════════════════════════════════════════════════════╗" -ForegroundColor
DarkGray
    Write-Host "  ║           WINDOWS 11 OPTIMIZER v2.0             ║" -ForegroundColor Green
    Write-Host "  ╚══════════════════════════════════════════════════════════╝" -ForegroundColor
DarkGray
    Write-Host ""
    # Show applied status
    $layers = @("MINIMAL","MODERATE","ULTIMATE","GOD MODE")
    foreach ($l in $layers) {
        $applied = $globalState.applied -contains $l
        $mark = if ($applied) { "[x]" } else { "[ ]" }
        $color = if ($applied) { "Green" } else { "DarkGray" }
        Write-Host "    $mark  $l" -ForegroundColor $color
    }
    Write-Host ""
    Write-Host "  [1] Apply MINIMAL    [2] Apply MODERATE" -ForegroundColor Cyan
    Write-Host "  [3] Apply ULTIMATE   [4] Apply GOD MODE" -ForegroundColor Cyan
    Write-Host "  [5] ROLLBACK         [6] SYSTEM STATUS" -ForegroundColor Yellow
    Write-Host "  [7] VERIFY SYSTEM    [8] SNAPSHOT NOW" -ForegroundColor Yellow
    Write-Host "  [Q] QUIT" -ForegroundColor DarkGray
    Write-Host ""
    Write-Host "  > Choose: " -NoNewline -ForegroundColor Green
}

# Main event loop
while ($true) {
    Show-MainMenu
    $choice = $Host.UI.RawUI.ReadKey("NoEcho,IncludeKeyDown").Character.ToString()
    switch ($choice) {
        "1" { Invoke-MinimalLayer  -State $globalState }
        "2" { Invoke-ModerateLayer -State $globalState }
        "3" { Invoke-UltimateLayer -State $globalState }
```

```
        "4" { Invoke-GodModeLayer  -State $globalState }
        "5" { Show-RollbackMenu    -State $globalState }
        "6" { Show-StatusPanel }
        "7" { Invoke-SystemVerification }
        "8" { Export-AllSnapshots }
        {$_ -in "q","Q"} { Write-Log -Level INFO -Msg "Session ended cleanly"; exit 0 }
    }
}
```

12 | POWERSHELL MODULE PATTERNS

LAYER MODULE STRUCTURE PATTERN

```
# Layer_Minimal.psm1 — Pattern for all layer modules

$LAYER_NAME = "MINIMAL"
$TWEAK_LIST = @()  # populated by Get-MinimalTweakList

function Invoke-MinimalLayer {
    param([PSCustomObject]$State, [switch]$DryRun)

    # Guard: check if already applied
    if ($State.applied -contains $LAYER_NAME) {
        Write-Host "  [WARN] $LAYER_NAME already applied. Re-apply? [Y/N]" -ForegroundColor Yellow
        $confirm = $Host.UI.RawUI.ReadKey("NoEcho,IncludeKeyDown").Character.ToString().ToUpper()
        if ($confirm -ne "Y") { return }
    }

    # Snapshot before starting
    Export-AllSnapshots
    Write-Log -Level INFO -Msg "Starting $LAYER_NAME layer"

    # Session-level consent state
    $globalRunAll = [ref]$false

    # Execute each tweak via master pattern
    Invoke-Tweak -TweakNumber 1 -Layer $LAYER_NAME `
        -TweakName "Disable DiagTrack Telemetry Service" `
        -Summary "Windows runs a background service..." `
        -WhatChanges "Stops + disables 1 service: DiagTrack" `
        -Reversible "YES — sc config DiagTrack start=auto" `
        -RiskLevel "ZERO" `
        -DryRun:$DryRun `
        -GlobalRunAll $globalRunAll `
        -ScriptBlock {
            sc.exe stop DiagTrack 2>$null
            sc.exe config DiagTrack start=disabled 2>$null
        } `
        -RollbackCmd "sc.exe config DiagTrack start=auto; sc.exe start DiagTrack"

    # ... repeat for all 16 tweaks ...

    # Mark as applied
    $State.applied += $LAYER_NAME
    $State | ConvertTo-Json | Set-Content "$PSScriptRoot\config\applied_layers.json"
    Write-Log -Level SUCCESS -Msg "$LAYER_NAME layer completed"
    Write-Host "`n  [DONE] $LAYER_NAME complete. Reboot recommended." -ForegroundColor Green
}

function Undo-MinimalLayer {
    $rollbackScript = "$PSScriptRoot\rollback\rollback_minimal.ps1"
    if (Test-Path $rollbackScript) { & $rollbackScript }
    else { Write-Host "  [WARN] No rollback script found." -ForegroundColor Yellow }
}

Export-ModuleMember -Function Invoke-MinimalLayer, Undo-MinimalLayer
```

13 | STATE MANAGEMENT & CONFIG SCHEMA

applied_layers.json SCHEMA

```
{
  "schemaVersion": "2.0",
  "machine": "DESKTOP-ABC123",
  "applied": ["MINIMAL", "MODERATE"],
  "appliedAt": {
    "MINIMAL":  "2024-01-15T12:43:00",
    "MODERATE": "2024-01-16T09:11:00"
  },
  "skippedTweaks": {
    "MINIMAL":  [3, 11],
    "MODERATE": [8]
  },
  "dryRunOnly": false,
  "lastVerification": {
    "timestamp":  "2024-01-16T09:30:00",
    "allPassed":  true,
    "failedTests": []
  },
  "snapshotPath": "C:\\WinOptimizer\\snapshots",
  "restorePoints": [
    { "layer": "MINIMAL",   "description": "pre-minimal-20240115",   "created": true },
    { "layer": "MODERATE",  "description": "pre-moderate-20240116",  "created": true }
  ]
}
```

user_prefs.json SCHEMA

```
{
  "colorTheme": "dark",         // dark | light | high-contrast
  "verbosity": "normal",        // quiet | normal | verbose
  "autoSnapshot": true,         // create snapshot before each layer
  "requireDiskImage": ["ULTIMATE", "GOD MODE"],
  "skipList": [],               // tweak IDs to always skip
  "defaultDryRun": false,
  "observationReminderHours": 48
}
```

protected_services.json SCHEMA

```
{
  "neverTouch": [
    "WinDefend", "mpssvc", "wuauserv", "TrustedInstaller",
    "RpcSs", "lsass", "winmgmt", "ShellHWDetection",
    "EventLog", "CryptSvc", "DcomLaunch", "PlugPlay"
  ],
  "neverRemoveAppx": [
    "Microsoft.WindowsStore",
    "Microsoft.UI.Xaml.*",
    "Microsoft.WindowsTerminal",
    "Microsoft.DesktopAppInstaller",
    "Microsoft.WindowsSecurity*",
    "Microsoft.Win32WebViewHost"
  ]
}
```

14 | LOGGING & AUDIT TRAIL SYSTEM

LOGGER MODULE (Logger.psm1)

```
function Write-Log {
    param(
        [ValidateSet("INFO","SUCCESS","SKIP","DRYRUN","WARN","ERROR")]
        [string]$Level,
        [string]$Msg
    )
    $ts = Get-Date -Format "yyyy-MM-dd HH:mm:ss"
    $entry = "[$ts] [$($Level.PadRight(7))] $Msg"

    # Append to session log (append-only, never overwrites)
    $logFile = "$PSScriptRoot\logs\session_$(Get-Date -f yyyyMMdd_HHmm).log"
    Add-Content -Path $logFile -Value $entry -Encoding UTF8

    # Console color map
    $colorMap = @{
        "INFO"    = "Cyan"
        "SUCCESS" = "Green"
        "SKIP"    = "DarkGray"
        "DRYRUN"  = "Blue"
        "WARN"    = "Yellow"
        "ERROR"   = "Red"
    }
    Write-Host "  $entry" -ForegroundColor $colorMap[$Level]
}

# Sample log output:
# [2024-01-15 12:43:01] [INFO   ] Session started
# [2024-01-15 12:43:05] [INFO   ] Starting MINIMAL layer
# [2024-01-15 12:43:07] [SUCCESS] APPLIED: Disable DiagTrack Telemetry Service
# [2024-01-15 12:43:09] [SKIP   ] SKIPPED: Disable Xbox Game Bar
# [2024-01-15 12:43:11] [SUCCESS] APPLIED: Disable Background App Access
# [2024-01-15 12:44:02] [SUCCESS] MINIMAL layer completed
```

FIELD	FORMAT	EXAMPLE
Timestamp	yyyy-MM-dd HH:mm:ss	2024-01-15 12:43:01
Level	INFO|SUCCESS|SKIP|WARN|ERROR	SUCCESS
Message	Free text — action taken	APPLIED: Disable DiagTrack
File	Per-session log file	session_20240115_1243.log
Encoding	UTF-8, LF line endings	Compatible with all log parsers

15 | ERROR HANDLING ARCHITECTURE

Every tweak is wrapped in try/catch. Failures are non-fatal by default — the loop continues after offering rollback. Only user-facing critical failures (admin check, disk image missing for God Mode) halt execution entirely.

ERROR CATEGORY	BEHAVIOR	USER ACTION
Admin check fails	HALT: refuse to start any layer	Re-run as Administrator
Restore point fails	WARN + ask to continue or abort	User choice to proceed
Single tweak fails	SHOW error + offer rollback of that tweak	Y=rollback, N=continue
Service not found	SKIP silently + log as INFO	No action needed
AppX remove fails	WARN + log + continue to next tweak	Manual Store reinstall
Firewall rule exists	Log as WARN, skip duplicate rule	No action needed
JSON config corrupted	Rebuild from template + log WARN	Review rebuilt config
CTRL+C / Force exit	Save state file + log session end	Resume with state intact
Rollback script missing	WARN + show manual inverse commands	Run displayed commands

```
# Error handling wrapper — used in every tweak
try {
    # Pre-condition check
    if (-not (Test-Path $targetPath)) {
        Write-Log -Level WARN -Msg "Target not found: $targetPath — skipping"
        return  # graceful skip, not a hard fail
    }

    # Execute tweak
    Invoke-TweakCommands

    Write-Log -Level SUCCESS -Msg "APPLIED: $TweakName"

} catch [System.UnauthorizedAccessException] {
    Write-Log -Level ERROR -Msg "ACCESS DENIED on $TweakName — run as Admin"
    throw  # re-throw — this IS a hard fail

} catch [System.Management.Automation.CommandNotFoundException] {
    Write-Log -Level WARN -Msg "Command not found for $TweakName — skipping"
    # don't re-throw — continue loop

} catch {
    Write-Log -Level ERROR -Msg "FAILED: $TweakName — $_"
    Write-Host "  [FAIL] $TweakName" -ForegroundColor Red
    Write-Host "  Rollback this tweak? [Y/N] " -NoNewline -ForegroundColor Yellow
    $rb = $Host.UI.RawUI.ReadKey("NoEcho,IncludeKeyDown").Character.ToString().ToUpper()
    if ($rb -eq "Y") { Invoke-RollbackSingle -TweakId $TweakNumber -Layer $Layer }
    # continue loop regardless
}
```

16 | BUILD, PACKAGING & DISTRIBUTION

DISTRIBUTION STRATEGY

The tool is distributed as a single folder containing the main .ps1 and a modules/ subdirectory. There is no installer. No internet required after download. No third-party dependencies. This is a deliberate design choice — security-conscious users should never run arbitrary internet-fetched scripts.

DISTRIBUTION FORMAT	DESCRIPTION	PROS
ZIP archive	WinOptimizer_v2.0.zip from GitHub releases	Easy to verify hash, no installer
Single PS1 (minified)	Optional: all modules inlined	One file, easiest to audit
Winget package	Future: winget install WinOptimizer	Easy updates, signed
Chocolatey	choco install winoptimizer	Enterprise deployment

PRE-RELEASE CHECKLIST

[ ] All 46 disclaimer summaries reviewed for plain English clarity

[ ] Dry-run mode tested on clean Windows 11 VM — no actual changes

[ ] Each layer tested individually on Win11 Home, Pro, and Enterprise

[ ] Rollback scripts tested: apply layer → rollback → verify baseline matches

[ ] All protected services confirmed never touched by any tweak

[ ] Windows Update run successfully after each layer on test machine

[ ] PowerShell 5.1 and 7.x both tested

[ ] CTRL+C mid-run tested — state file saves correctly

[ ] Admin check tested: confirm tool refuses to run without elevation

[ ] SHA256 hash generated for the release ZIP and published alongside

17 | DEPLOYMENT RUNBOOK

STEP-BY-STEP FIRST RUN

STEP	ACTION	COMMAND / METHOD
1	Verify Windows version	winver — confirm 21H2 or newer
2	Verify admin rights	Run terminal as Administrator
3	Set execution policy	Set-ExecutionPolicy RemoteSigned -Scope CurrentUser
4	Download and extract ZIP	GitHub releases → verify SHA256
5	Review disclaimer for desired layer	Read this document Sections 7-10
6	Create disk image (mandatory for Ultimate+)	Macrium Reflect Free or Clonezilla
7	Run tool	cd WinOptimizer; .\WinOptimizer.ps1
8	Choose layer from main menu	Start with [1] MINIMAL
9	Review each disclaimer, press Y/N/A/D	Take your time — no rush
10	Reboot after each layer	Standard Windows reboot
11	Observe for 48-72 hours	Normal usage — watch for issues
12	Run verification [7] from main menu	All checks should PASS
13	Proceed to next layer when ready	Or stop at current layer permanently

ROLLBACK RUNBOOK (If Something Breaks)

STEP	ACTION	HOW
1	Open tool as Admin	.\WinOptimizer.ps1
2	Select [5] ROLLBACK from main menu	Keyboard: press 5
3	Choose layer to rollback	Most recent layer first
4	Confirm rollback (type YES)	Tool creates restore point first
5	Reboot	Standard reboot
6	Run [7] VERIFY from main menu	Confirm baseline restored
IF TOOL WON'T START:	Run rollback script directly	powershell .\rollback\rollback_[layer].ps1
IF SCRIPT FAILS:	Use System Restore (GUI)	rstrui.exe
IF RESTORE FAILS:	Restore disk image	Boot from Macrium/Clonezilla media

18 | MAINTENANCE & UPDATE PROCEDURES

ONGOING MAINTENANCE TASKS

TASK	FREQUENCY	PROCEDURE
Review applied tweaks	Monthly	Run [7] Verify — confirm all still applied
Windows major update	After each	Re-run verify — WU can re-enable services
Add new telemetry IPs	Quarterly	Update firewall rule list in Layer_Ultimate
Review disclaimer text	Per release	Ensure plain English, <100 words, accurate
Test rollback scripts	Per release	VM test: apply → rollback → verify clean
Update protected services	Per release	Check if new Win11 build adds critical svcs
Audit new scheduled tasks	Monthly	schtasks /query — new telemetry tasks?
Check OEM app list	Quarterly	New OEM bloat may appear with driver updates

WINDOWS FEATURE UPDATE RECOVERY

Major Windows feature updates (23H2, 24H2, etc.) can reset some Group Policy registry values and re-enable some services. After any Windows feature update, run the verification suite from the tool's main menu. For services that got re-enabled, you can simply re-run the affected layer — the tool detects already-applied state and asks before re-applying.

KNOWN WINDOWS BUILD VARIATIONS

WIN11 BUILD	KNOWN DIFFERENCES
21H2 (original)	No Copilot, no Widgets in first builds, fewer CEIP tasks
22H2	Widgets added — ensure Widgets tweak applied
23H2	Copilot added — ensure Copilot tweak applied
24H2	Recall feature in some hardware SKUs — verify Recall tweak
Future builds	Re-audit CEIP task list — Microsoft adds new ones with features

DOCUMENT COMPLETE

DOCUMENT	CONTENTS
Doc 1 — Architecture	System design, layer map, component hierarchy, state machine
Doc 2 — Implementation	Module code, commands, rollback engine, TUI design
Doc 3 — Testing	Verification suite, checklists, regression tests, performance tracking
Doc 4 — This Document	Tech stack, disclaimer system, all 46 disclaimer cards, full implementation

Every disclaimer card in this document represents the exact text a user sees before any tweak executes. The consent-first philosophy is the defining feature of this tool. Build it this way. Don't compromise on it.